

实验进阶3 银行业务数据封装

难度（最高三星）：★★★ 建议学时（2-4学时）：4学时

实验说明

学习地图

某银行是一家小型银行企业，有十多万客户。现在要为该银行开发一套银行业务系统，通过计算机对银行日常的业务进行处理，以保证数据的安全性，提高工作效率。

本系统的功能结构图如图1所示。

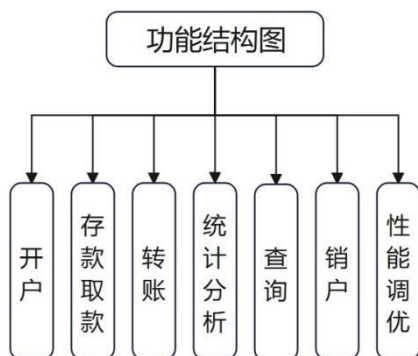


图1 银行业务系统功能结构图

本项目实验案例集要求根据银行业务需求设计出符合第3范式的数据库结构，使用SQL语言创建数据库和表，并添加表约束，进行数据的增、删、改、查，运用触发器、事务、视图和存储过程，按照银行的业务需求实现各项业务（包括日常存款、取款和转账等），同时为了适应日益增长的客户需求，要对数据库进行性能调优。

本项目将完成一个典型的数据库管理系统——银行业务系统的项目开发，要求完成的项目能够满足用户需求中的功能要求。共分为5个实验进阶、15个任务，按照难度等级分为初级、中级、高级，如图2所示。



图2 银行业务系统数据库

整个项目实验案例集将涉及openGauss数据库多个知识点和技能，如图3所示。

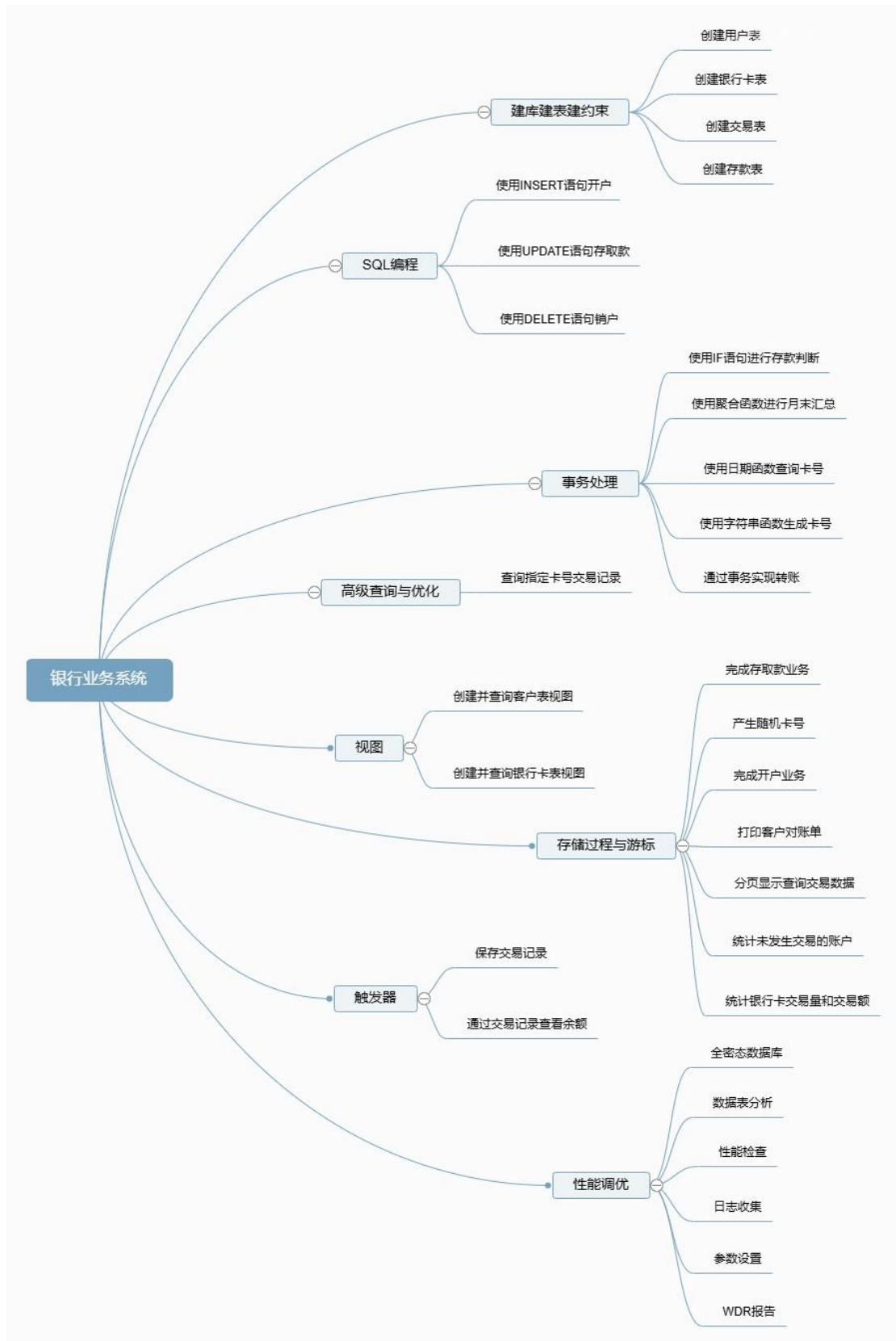


图3 银行业务系统知识点和技能

实验环境准备

具体开发环境如下：

- (1) 数据库：openGauss 5.0.2
- (2) 可视化开发工具：Data Studio
- (3) 终端工具：FinalShell，其安装和建立连接步骤见[网址](#)。
- (4) 数据库创建：该部分请参照实验进阶1【[银行业务系统数据库表设计](#)】中任务完成，

或直接[下载相关代码](#)（如下）并运行。

```
CREATE DATABASE bankdb;

CREATE USER bank_admin WITH PASSWORD'Bank@1234';

GRANT ALL PRIVILEGES ON DATABASE bankdb TO bank_admin;


CREATE SCHEMA bank_schema;
SET SEARCH_PATH TO bank_schema


CREATE TABLE userInfo
(
    customerID SERIAL PRIMARY KEY,
    customerName CHAR(8) NOT NULL,
    PID CHAR(18) NOT NULL,
    telephone CHAR(20) NOT NULL,
    address VARCHAR(50)
);


CREATE TABLE cardInfo
(
    cardID CHAR(17) PRIMARY KEY,
    curlID VARCHAR(10) DEFAULT 'RMB' NOT NULL,
    savingID INT NOT NULL DEFAULT 1,
    openDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
openMoney NUMERIC(18,2) NOT NULL,
balance NUMERIC(18,2) NOT NULL,
pass CHAR(6) NOT NULL DEFAULT '888888',
IsReportLoss VARCHAR(3) NOT NULL DEFAULT 'F',
customerID INT NOT NULL
);

CREATE TABLE tradeInfo
(
tradeDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
tradeType CHAR(6) NOT NULL,
cardID CHAR(19) NOT NULL,
tradeMoney NUMERIC(18,2) NOT NULL,
remark TEXT
);

CREATE TABLE deposit
(
savingID SERIAL PRIMARY KEY,
savingName VARCHAR(20) NOT NULL,
descrip VARCHAR(50)
);

ALTER TABLE userInfo
ADD CONSTRAINT chk_PID CHECK (CHAR_LENGTH(CAST(PID AS TEXT)) = 18),
ADD CONSTRAINT chk_telephone CHECK
(CHAR_LENGTH(REGEXP_REPLACE(CAST(telephone AS TEXT), '[^0-9]', '', 'g')) = 11);

ALTER TABLE cardInfo
ADD CONSTRAINT ck_cardid CHECK (
SUBSTRING(cardID, 1, 8) = '10103576'
AND SUBSTRING(cardID, 9) ~ '^ [0-9]{8}$'
),
ADD CONSTRAINT CK_openMoney CHECK (openMoney >= 1),
```

```
ADD CONSTRAINT CK_balance CHECK (balance >=1),
ADD CONSTRAINT CK_pass CHECK (LENGTH(pass) = 6 AND pass ~ '^[0-9]+$'),
ADD CONSTRAINT FK_customerID FOREIGN KEY(customerID) REFERENCES
userInfo(customerID),
ADD CONSTRAINT FK_savingID FOREIGN KEY(savingID) REFERENCES deposit(savingID);

ALTER TABLE tradeInfo
ADD CONSTRAINT CK_tradeType CHECK(tradeType IN ('Credit', 'Debit')),
ADD CONSTRAINT CK_tradeMoney CHECK(tradeMoney > 0);
```

(5) 数据库数据录入：该部分请参照实验进阶2【[银行业务数据录入与修改](#)】中任务完成，或直接[下载相关代码](#)（如下）并运行。

```
INSERT INTO deposit (savingName, descrip) VALUES ('Current Deposit', 'Interest is
calculated based on the deposit date');

INSERT INTO deposit (savingName, descrip) VALUES ('1Y Fixed', '1 year term');
INSERT INTO deposit (savingName, descrip) VALUES ('2Y Fixed', '2 year term');
INSERT INTO deposit (savingName, descrip) VALUES ('3Y Fixed', '3 year term');
INSERT INTO deposit (savingName) VALUES ('Flex Deposit');
INSERT INTO deposit (savingName, descrip) VALUES ('1Y Savings', '1 year term');
INSERT INTO deposit (savingName, descrip) VALUES ('2Y Savings', '2 year term');
INSERT INTO deposit (savingName, descrip) VALUES ('3Y Savings', '3 year term');
INSERT INTO deposit (savingName, descrip) VALUES ('5Y Int Only', 'monthly int');

INSERT INTO userInfo(customerid, customerName, PID, telephone, address)
VALUES(1, 'John', '11000000000000000001', '13900000001', 'Nanjing City, Jiangsu
Province');

INSERT INTO userInfo(customerid, customerName, PID, telephone)
VALUES(2, 'Jane', '11000000000000000002', '13900000002');

INSERT INTO userInfo(customerid, customerName, PID, telephone)
VALUES(3, 'Michael', '11000000000000000003', '13900000003');

INSERT INTO userInfo(customerid, customerName, PID, telephone)
VALUES(4, 'Emily', '11000000000000000004', '13900000004');

INSERT INTO userInfo(customerid, customerName, PID, telephone)
VALUES(5, 'David', '11000000000000000005', '13900000005');
```

```
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(6,'Sarah','110000000000000006','13900000006');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(7,'Robert','110000000000000007','13900000007');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(8,'Jessica','110000000000000008','13900000008');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(9,'William','110000000000000009','13900000009');
INSERT INTO userInfo(customerid,customerName,PID,telephone)
VALUES(10,'Olivia','110000000000000010','13900000010');
```

```
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000001', 1, 1000, 1000, 1);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000002', 2, 1, 1, 2);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000003', 2, 1, 1, 3);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000004', 2, 1, 1, 4);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000005', 3, 500, 500, 5);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000006', 3, 300, 300, 6);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000007', 3, 200, 200, 7);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000008', 3, 150, 150, 8);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000009', 3, 100, 100, 9);
INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
VALUES ('1010357600000010', 3, 80, 80, 10);
```

实验任务

任务描述

本次实验进阶3，主要是实现对银行业务数据系统的封装，共计四个任务，包括：

任务1：使用视图多表查询；

任务2：使用触发器实现数据完整；

任务3：使用存储过程操作数据；

任务4：使用事务进行转账。

学习目标

通过这四项任务能够完成数据的视图、存储过程、触发器等封装技术使用。

任务准备

前置知识

本实验案例，需提前学习openGauss的基本概念、作用及基础操作。可通过如下途径进行学习：

- 数字教材：《[数据库原理与应用：基于openGauss](#)》
- 技术文档：[openGauss白皮书](#)

任务实施

任务 1：使用视图多表查询

本实验的具体操作可参见视频。

为了向客户显示信息友好,查询各表要求字段全为中文字段名。

创建客户信息表视图,方便管理员直观查看每个客户的银行卡信息。

```
DROP VIEW IF EXISTS vw_userInfo;  
CREATE VIEW vw_userInfo  
AS  
SELECT  
    customerID AS "客户编号",  
    customerName AS "开户名",  
    PID AS "身份证号",  
    telephone AS "电话号码",  
    address AS "居住地址"  
FROM userInfo;
```

查询客户信息表视图,如图1.1所示。

```
SELECT * FROM vw_userInfo;
```

	客户编号	开户名	身份证号	电话号码	居住地址
1	1	John	11000000000000000001	13900000001	江苏省南京市
2	2	Jane	11000000000000000002	13900000002	[NULL]
3	3	Michael	11000000000000000003	13900000003	[NULL]
4	4	Emily	11000000000000000004	13900000004	[NULL]
5	5	David	11000000000000000005	13900000005	[NULL]
6	6	Sarah	11000000000000000006	13900000006	[NULL]
7	7	Robert	11000000000000000007	13900000007	[NULL]
8	8	Jessica	11000000000000000008	13900000008	[NULL]
9	9	William	11000000000000000009	13900000009	[NULL]
10	10	Olivia	11000000000000000010	13900000010	[NULL]

图1.1 用户信息视图

创建银行卡信息视图，方便管理员直观查看每个客户的银行卡信息。

```
CREATE VIEW vw_cardInfo
AS
SELECT
    c.cardID AS "卡号",
    u.customerName AS "客户姓名",
    c.curID AS "货币种类",
    d.savingName AS "存款类型",
    c.openDate AS "开户日期",
    c.balance AS "余额",
    c.pass AS "密码",
    CASE WHEN c.IsReportLoss = '否' THEN '正常' ELSE '挂失' END AS "是否挂失"
FROM cardInfo c
JOIN deposit d ON c.savingID = d.savingID
JOIN userInfo u ON c.customerID = u.customerID;
```

查询银行卡信息视图，如图1.2所示。

```
SELECT * FROM vw_cardInfo;
```

	卡号	客户姓名	货币种类	存款类型	开户日期	余额	密码	是否挂失
1	1010357600000001	John	RMB	活期	2024-08-26 15:32:32	1000.00	888888	正常
2	1010357600000002	Jane	RMB	定期一年	2024-08-26 15:32:32	1.00	888888	正常
3	1010357600000003	Michael	RMB	定期一年	2024-08-26 15:32:32	1.00	888888	正常
4	1010357600000005	David	RMB	定期二年	2024-08-26 15:32:32	500.00	888888	正常
5	1010357600000006	Sarah	RMB	定期二年	2024-08-26 15:32:32	300.00	888888	正常
6	1010357600000008	Jessica	RMB	定期二年	2024-08-26 15:32:32	150.00	888888	正常
7	1010357600000009	William	RMB	定期二年	2024-08-26 15:32:32	100.00	888888	正常
8	1010357600000010	Olivia	RMB	定期二年	2024-08-26 15:32:32	80.00	123456	正常
9	1010357600000004	Emily	RMB	定期一年	2024-08-26 15:32:32	1.00	123123	正常
10	1010357600000007	Robert	RMB	定期二年	2024-08-26 15:32:32	200.00	888888	挂失

图1.2 银行卡信息视图

任务 2：使用触发器实现数据完整

本实验的具体操作可参见视频。

用户John的卡号（1010357600000001）取款900元，用户Jane的卡号

(1010357600000002) 存款5000元, 要求保存交易记录, 以便客户查询和银行业务统计。

说明: 当存钱或取钱 (如900元) 时候, 会向交易信息表 (tradeInfo) 中添加一条交易记录, 同时应更新银行卡信息表 (cardInfo) 中的现有余额 (如增加或减少900元) 。

(1) 向交易信息表 (tradeInfo) 插入交易记录。

```
INSERT INTO tradeInfo(tradeType, cardID, tradeMoney)
VALUES('支取', '1010357600000001', 900);

INSERT INTO tradeInfo(tradeType, cardID, tradeMoney)
VALUES('存入', '1010357600000002', 5000);
```

(2) 查看交易信息表 (tradeInfo) 是否被成功插入数据。

```
SELECT * FROM tradeinfo;
```

(3) 更新银行卡信息表 (cardInfo) 中的现有余额。

```
UPDATE cardInfo
SET balance = balance - 900
WHERE cardID = '1010357600000001';

UPDATE cardInfo
SET balance = balance + 5000
WHERE cardID = '1010357600000002';
```

(4) 查看银行卡信息表 (cardInfo) 中的数据是否被成功修改, 如图2.1所示。

```
SELECT * FROM cardinfo;
```

9	1010357600000001	RMB	1	2024-08-08 15:34:09	1000.00	100.00	888888	否	1
10	1010357600000002	RMB	2	2024-08-08 15:40:08	1.00	5001.00	888888	否	2

图2.1 查看银行卡信息表

(5) 多次进行该银行业务。

```
-- 1010357600000003 存入300

INSERT INTO tradeInfo(tradeType, cardID, tradeMoney)
```

```
VALUES('存入', '1010357600000003', 300);

UPDATE cardInfo
SET balance = balance + 300
WHERE cardID = '1010357600000003';

-- 1010357600000004 存入1000
INSERT INTO tradeInfo(tradeType, cardID, tradeMoney)
VALUES('存入', '1010357600000004', 1000);
UPDATE cardInfo
SET balance = balance + 1000
WHERE cardID = '1010357600000004';

-- 1010357600000002 支取1900
INSERT INTO tradeInfo(tradeType, cardID, tradeMoney)
VALUES('支取', '1010357600000002', 1900);
UPDATE cardInfo
SET balance = balance - 1900
WHERE cardID = '1010357600000002';
```

通过上面的例子可以看出，手动维护银行卡交易和账户余额数据效率低下且容易出错，可以用触发器在插入交易记录的同时自动更新银行卡的余额，确保交易记录和账户余额始终保持一致。

```
CREATE OR REPLACE FUNCTION update_balance()
RETURNS TRIGGER AS $$
DECLARE
    v_current_balance NUMERIC(18,2);
BEGIN
    IF NEW.tradeType = '存入' THEN
        UPDATE cardInfo
        SET balance = balance + NEW.tradeMoney
        WHERE cardID = NEW.cardID;
    ELSIF NEW.tradeType = '支取' THEN
        SELECT balance INTO v_current_balance
```

```

FROM cardInfo
WHERE cardID = NEW.cardID;

IF v_current_balance >= NEW.tradeMoney + 1 THEN
    UPDATE cardInfo
    SET balance = balance - NEW.tradeMoney
    WHERE cardID = NEW.cardID;
ELSE
    RAISE EXCEPTION '余额不足';
END IF;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_balance
BEFORE INSERT ON tradeInfo
FOR EACH ROW
EXECUTE PROCEDURE update_balance();

```

我们再尝试插入一些交易记录并查看银行卡信息表，如图2.2所示。

```

INSERT INTO tradeInfo(tradeType, cardID, tradeMoney)
VALUES('存入', '1010357600000005', 2000);

INSERT INTO tradeInfo(tradeType, cardID, tradeMoney)
VALUES('支取', '1010357600000006', 200);

```

9	1010357600000005	RMB	3	2024-08-08 15:40:27	500.00	2500.00	888888	否	5
10	1010357600000006	RMB	3	2024-08-08 15:40:33	300.00	100.00	888888	否	6

图2.2 查看银行卡信息表

可以发现此时在不更新账户余额的情况下，账户余额会随着交易的正常执行而变化。

任务 3：使用存储过程操作数据

本实验的具体操作可参见视频。

1. 完成存款或取款业务

根据银行卡号和交易金额，实现银行卡的存款和取款业务，每一笔存款、取款业务都要记入银行交易账，并同时更新客户的存款余额。需要提供一个集中的、可靠的界面来管理银行卡的存取款操作，如果存取款不成功会有错误提醒。

```
CREATE OR REPLACE PROCEDURE deposit_withdraw(  
    p_cardid CHAR(19),  
    p_money NUMERIC(18,2),  
    p_trade_type CHAR(4)  
)  
AS  
    v_balance NUMERIC(18,2);  
BEGIN  
    -- 检查卡号是否存在  
    IF NOT EXISTS (SELECT 1 FROM cardInfo WHERE cardID = p_cardid) THEN  
        RAISE EXCEPTION '卡号不存在';  
    END IF;  
  
    -- 检查交易类型是否合法  
    IF p_trade_type NOT IN ('存入', '支取') THEN  
        RAISE EXCEPTION '输入不符合规范';  
    END IF;  
  
    -- 执行存取款操作  
    IF p_trade_type = '存入' THEN  
        -- 存款  
        UPDATE cardInfo  
        SET balance = balance + p_money  
        WHERE cardID = p_cardid;
```

```
-- 记录交易信息
INSERT INTO tradeInfo (cardID, tradeType, tradeMoney, remark)
VALUES (p_cardid, '存入', p_money, NULL);
ELSIF p_trade_type = '支取' THEN
    -- 取款
    SELECT balance INTO v_balance FROM cardInfo WHERE cardID = p_cardid;
    IF v_balance < p_money THEN
        RAISE EXCEPTION '金额不足';
    END IF;
    UPDATE cardInfo
    SET balance = balance - p_money
    WHERE cardID = p_cardid;

    -- 记录交易信息
    INSERT INTO tradeInfo (cardID, tradeType, tradeMoney, remark)
    VALUES (p_cardid, '支取', p_money, NULL);
END IF;
END;
/
```

调用存储过程。

```
CALL deposit_withdraw('1010357600000001', 1000.00, '存入'); --正常执行
CALL deposit_withdraw('1010357600000002', 500.00, '支取'); --正常执行
CALL deposit_withdraw('1010357600000011', 500.00, '支取'); --错误的卡号
CALL deposit_withdraw('1010357600000002', 5000.00, '支取'); --支取的金额超出余额
CALL deposit_withdraw('1010357600000002', 500.00, '取款'); --错误的输入类型
```

查询结果为如下。

执行成功

执行成功

ERROR: 卡号不存在

ERROR: 金额不足

ERROR: 输入不符合规范

2. 产生随机卡号

创建存储过程会产生8位随机数字，与前8位固定的数字“10103576”连接，生成一个由 16 位数字组成的银行卡号，并输出。

```
CREATE OR REPLACE PROCEDURE usp_randCardID(OUT randCardID VARCHAR(17))
AS
DECLARE
    r1 NUMERIC(15, 8);
    r2 NUMERIC(15, 8);
    tempStr1 CHAR(10);
    tempStr2 CHAR(10);
BEGIN
    -- 生成整数随机数种子，并确保其在可接受的范围内
    r1 := CAST(RANDOM() * 9999999 AS BIGINT); -- 确保随机数长度不超过7位
    r2 := CAST(RANDOM() * 9999999 AS BIGINT);
    -- 转换为字符串
    tempStr1 := LPAD(CAST(r1 AS VARCHAR), 10, '0'); -- 保证字符串长度为10，不足部分左侧
    补零
    tempStr2 := LPAD(CAST(r2 AS VARCHAR), 10, '0');
    -- 组合成规定格式的卡号
    randCardID := '10103576' || SUBSTRING(tempStr1 FROM 1 FOR 4) ||
SUBSTRING(tempStr2 FROM 1 FOR 4);
END;
/
调用存储过程。
DO $$
DECLARE
    mycardID VARCHAR(17);
BEGIN
    mycardID := usp_randCardID(); -- 直接调用函数并赋值
    RAISE NOTICE '产生的随机卡号为: %', mycardID;
END $$;
```


查询结果如下。

产生的随机卡号为：10103576xxxxxxx。

3. 完成开户业务

在刚刚创建的存储过程的基础上新建一个存储过程，需要输入客户姓名、身份证号、电话号码、开户金额、存款类型，选择填写地址。

```
CREATE OR REPLACE PROCEDURE usp_openAccount(  
    IN p_customerName VARCHAR(8),  
    IN p_PID CHAR(18),  
    IN p_telephone CHAR(11),  
    IN p_openMoney NUMERIC(18,2),  
    IN p_savingName CHAR(8),  
    IN p_address VARCHAR(50) DEFAULT ''  
)  
AS  
DECLARE  
    v_cardID VARCHAR(19);  
    v_customerID INTEGER;  
    v_savingID INTEGER;  
BEGIN  
    -- 调用产生随机卡号的存储过程获得随机卡号  
    v_cardID := usp_randCardID();  
    WHILE EXISTS (SELECT * FROM cardInfo WHERE cardID = v_cardID)  
    LOOP  
        v_cardID := usp_randCardID();  
    END LOOP;  
  
    RAISE NOTICE '尊敬的客户,开户成功!系统为您产生的随机卡号为:', v_cardID;  
    RAISE NOTICE '开户日期%  开户金额:', CURRENT_DATE, p_openMoney;  
  
    -- 如果用户信息不存在,则插入新用户  
    IF NOT EXISTS (SELECT * FROM userInfo WHERE PID = p_PID) THEN
```

```
        LOOP
            -- 生成新的 customerID
            SELECT NEXTVAL('userinfo_customerid_seq') INTO v_customerID;
            -- 检查 customerID 是否存在
            IF NOT EXISTS (SELECT * FROM userInfo WHERE customerID = v_customerID)
THEN
                INSERT INTO userInfo (customerID, customerName, PID, telephone,
address)
                    VALUES (v_customerID, p_customerName, p_PID, p_telephone, p_address);
                EXIT;
            END IF;
        END LOOP;
    ELSE
        -- 获取现有的 customerID
        SELECT customerID INTO v_customerID FROM userInfo WHERE PID = p_PID;
    END IF;

    -- 查找存款类型ID
    SELECT savingID INTO v_savingID FROM deposit WHERE savingName =
p_savingName;
    IF v_savingID IS NULL THEN
        RAISE EXCEPTION '存款类型不正确,请重新输入!' USING ERRCODE = '22000';
    END IF;

    -- 插入新卡信息
    INSERT INTO cardInfo (cardID, savingID, openMoney, balance, customerID)
        VALUES (v_cardID, v_savingID, p_openMoney, p_openMoney, v_customerID);
    END;
/
```

调用存储过程。

```
CALL usp_openAccount('Emma', '110000000000000011', '13900000011', 1000.00, '活期', '辽宁省大连市');
```

查询结果为：尊敬的客户，开户成功！系统为您产生的随机卡号为：

10103576XXXXXXXXX 开户日期202X-XX-XX 开户金额: 1000.00

```
CALL usp_openAccount('Henry', '110000000000000012', '13900000012', 1.00, '定期一年');
```

查询结果如下。

尊敬的客户, 开户成功! 系统为您产生的随机卡号为: 10103576XXXXXXXXX 开户日期202X-XX-XX 开户金额: 1.00

4. 打印客户对账单

客户在银行办理业务的时候经常需要打印对账单, 即为某个特定的银行卡号打印指定时间内发生交易的对账单。

```
CREATE OR REPLACE PROCEDURE usp_CheckSheet(  
    IN p_cardID VARCHAR(19),  
    IN p_date1 TIMESTAMP DEFAULT NULL,  
    IN p_date2 TIMESTAMP DEFAULT NULL  
)  
AS  
DECLARE  
    v_cardid VARCHAR(19);  
    v_custName VARCHAR(20);  
    v_curName VARCHAR(20);  
    v_savingName VARCHAR(20);  
    v_openDate TIMESTAMP;  
    rec RECORD;  
BEGIN  
    SELECT c.cardID, c.curID, u.customerName, d.savingName, c.openDate  
    INTO v_cardID, v_curName, v_custName, v_savingName, v_openDate  
    FROM cardInfo c  
    JOIN userInfo u ON c.customerID = u.customerID  
    JOIN deposit d ON c.savingID = d.savingID  
    WHERE c.cardID = p_cardID;
```

```
RAISE NOTICE '卡号: %', v_cardID;
RAISE NOTICE '姓名: %', v_custName;
RAISE NOTICE '货币: %', v_curName;
RAISE NOTICE '存款类型: %', v_savingName;
RAISE NOTICE '开户日期: %年%月%日', TO_CHAR(v_openDate, 'YYYY'),
TO_CHAR(v_openDate, 'MM'), TO_CHAR(v_openDate, 'DD');
RAISE NOTICE ' ';
RAISE NOTICE '-----';

IF p_date1 IS NULL AND p_date2 IS NULL THEN
    FOR rec IN
        SELECT tradeDate AS "交易日", tradeType AS "类型", tradeMoney AS "交易金额",
remark AS "备注"
        FROM tradeInfo
        WHERE cardID = p_cardID
        ORDER BY tradeDate
    LOOP
        RAISE NOTICE '%', rec;
    END LOOP;
ELSE
    IF p_date2 IS NULL THEN
        p_date2 := CURRENT_TIMESTAMP;
    END IF;

    FOR rec IN
        SELECT tradeDate AS "交易日", tradeType AS "类型", tradeMoney AS "交易金额",
remark AS "备注"
        FROM tradeInfo
        WHERE cardID = p_cardID AND tradeDate BETWEEN p_date1 AND p_date2
        ORDER BY tradeDate
    LOOP
        RAISE NOTICE '%', rec;
    END LOOP;
```

```
END IF;  
END;  
/
```

调用存储过程，如图3.1所示。

```
CALL usp_CheckSheet('1010357600000002');  
CALL usp_CheckSheet('10103576000000001', '2024-08-25', '2024-08-27');
```

```
[NOTICE] 卡号: 10103576000000002  
[NOTICE] 姓名: Jane  
[NOTICE] 货币: RMB  
[NOTICE] 存款类型: 定期一年  
[NOTICE] 开户日期: 2024年08月08日  
[NOTICE]  
[NOTICE] -----  
[NOTICE] ("2024-08-08 15:53:02.45374",存入,5000.00,)  
[NOTICE] ("2024-08-08 16:05:04.042534",支取,1900.00,)  
[NOTICE] ("2024-08-08 16:54:02.374043",支取,500.00,)  
  
[NOTICE] 卡号: 10103576000000001  
[NOTICE] 姓名: John  
[NOTICE] 货币: RMB  
[NOTICE] 存款类型: 活期  
[NOTICE] 开户日期: 2024年08月08日  
[NOTICE]  
[NOTICE] -----  
[NOTICE] ("2024-08-08 15:52:59.189046",支取,900.00,)  
[NOTICE] ("2024-08-08 16:51:38.158797",存入,1000.00,)  
[NOTICE] ("2024-08-08 16:56:57.774065",存入,1000.00,)
```

图3.1 调用存储过程1

5. 分页显示查询交易数据

存储过程和游标用于实现查看交易记录的页数和每页显示的记录数，实现分页显示

(默认为输出第1页，每页10条记录)。

```
CREATE OR REPLACE PROCEDURE usp_pagingDisplay(  
    IN p_page INT = 1,  
    IN p_records INT = 10  
)  
AS  
DECLARE
```

```
v_offset INT;
v_tradeDate TEXT;
v_tradeType TEXT;
v_cardID TEXT;
v_tradeMoney NUMERIC;

-- 定义游标
CURSOR trade_cursor FOR
    SELECT tradeDate,
           tradeType,
           cardID,
           tradeMoney
    FROM tradeInfo
    ORDER BY tradeDate
    LIMIT p_records
    OFFSET v_offset;

BEGIN
    v_offset := (p_page - 1) * p_records;

    -- 打开游标
    OPEN trade_cursor;

    -- 循环读取游标数据
    LOOP
        FETCH trade_cursor INTO v_tradeDate, v_tradeType, v_cardID, v_tradeMoney;
        EXIT WHEN NOT FOUND;

        RAISE NOTICE '交易日期: %, 交易类型: %, 卡号: %, 交易金额: %', v_tradeDate,
v_tradeType, v_cardID, v_tradeMoney;
    END LOOP;

    -- 关闭游标
    CLOSE trade_cursor;

END;
```

/

调用存储过程，如图3.2所示。

```
CALL usp_pagingDisplay();
```

```
CALL usp_pagingDisplay(2,5);
```

```
[NOTICE] 交易日期: 2024-08-08 15:52:59.189046, 交易类型: 支取, 卡号: 1010357600000001, 交易金额: 900.00
[NOTICE] 交易日期: 2024-08-08 15:53:02.45374, 交易类型: 存入, 卡号: 1010357600000002, 交易金额: 5000.00
[NOTICE] 交易日期: 2024-08-08 16:03:15.151009, 交易类型: 存入, 卡号: 1010357600000003, 交易金额: 300.00
[NOTICE] 交易日期: 2024-08-08 16:03:53.412365, 交易类型: 存入, 卡号: 1010357600000004, 交易金额: 1000.00
[NOTICE] 交易日期: 2024-08-08 16:05:04.042534, 交易类型: 支取, 卡号: 1010357600000002, 交易金额: 1900.00
[NOTICE] 交易日期: 2024-08-08 16:38:41.058165, 交易类型: 存入, 卡号: 1010357600000005, 交易金额: 2000.00
[NOTICE] 交易日期: 2024-08-08 16:39:33.248524, 交易类型: 支取, 卡号: 1010357600000006, 交易金额: 200.00
[NOTICE] 交易日期: 2024-08-08 16:51:38.158797, 交易类型: 存入, 卡号: 1010357600000001, 交易金额: 1000.00
[NOTICE] 交易日期: 2024-08-08 16:54:02.374043, 交易类型: 支取, 卡号: 1010357600000002, 交易金额: 500.00
[NOTICE] 交易日期: 2024-08-08 16:56:57.774065, 交易类型: 存入, 卡号: 1010357600000001, 交易金额: 1000.00
```

```
[NOTICE] 交易日期: 2024-08-08 16:38:41.058165, 交易类型: 存入, 卡号: 1010357600000005, 交易金额: 2000.00
[NOTICE] 交易日期: 2024-08-08 16:39:33.248524, 交易类型: 支取, 卡号: 1010357600000006, 交易金额: 200.00
[NOTICE] 交易日期: 2024-08-08 16:51:38.158797, 交易类型: 存入, 卡号: 1010357600000001, 交易金额: 1000.00
[NOTICE] 交易日期: 2024-08-08 16:54:02.374043, 交易类型: 支取, 卡号: 1010357600000002, 交易金额: 500.00
[NOTICE] 交易日期: 2024-08-08 16:56:57.774065, 交易类型: 存入, 卡号: 1010357600000001, 交易金额: 1000.00
```

图3.2 调用存储过程2

6. 统计未发生交易的账户

用存储过程和游标查询、统计在指定时间段内没有发生交易的账户信息。

```
CREATE OR REPLACE PROCEDURE usp_getwithouttrade(
    IN date1 TIMESTAMP DEFAULT NULL,
    IN date2 TIMESTAMP DEFAULT NULL
)
AS
DECLARE
    v_Num INT;
    v_Amount DECIMAL(18,2);
    v_CustomerID BIGINT;
    v_CustomerName VARCHAR(50);
    v_PID VARCHAR(18);
    v_Telephone VARCHAR(20);
    v_Address VARCHAR(200);
```

```
-- 定义游标
CURSOR customer_cursor FOR
    SELECT DISTINCT u.customerID, u.customerName, u.PID, u.telephone, u.address
    FROM userInfo u
    JOIN cardInfo c ON u.customerID = c.customerID
    WHERE c.cardID NOT IN (SELECT cardID FROM tradeInfo WHERE tradeDate BETWEEN
date1 AND date2);

BEGIN
    -- 处理默认日期值
    IF date1 IS NULL THEN
        date1 := TO_TIMESTAMP(DATE_TRUNC('month', CURRENT_TIMESTAMP)::DATE, 'YYYY-
MM-DD HH24:MI:SS');
    END IF;

    IF date2 IS NULL THEN
        date2 := CURRENT_TIMESTAMP;
    END IF;

    -- 查询未发生交易的客户数和总余额
    SELECT COUNT(customerID), SUM(balance) INTO v_Num, v_Amount
    FROM cardInfo
    WHERE cardID NOT IN (SELECT cardID FROM tradeInfo WHERE tradeDate BETWEEN
date1 AND date2);

    RAISE NOTICE '统计未发生交易的客户';
    RAISE NOTICE '-----';
    RAISE NOTICE '客户人数: %, 客户总余额: %', v_Num, v_Amount;

    -- 打开游标
    OPEN customer_cursor;

    -- 循环读取游标数据
```



```
LOOP
    FETCH customer_cursor INTO v_CustomerID, v_CustomerName, v_PID, v_Telephone,
v_Address;

    EXIT WHEN NOT FOUND;

    RAISE NOTICE '客户号: %, 客户姓名: %, 身份证号: %, 电话: %, 地址: %',
        v_CustomerID, v_CustomerName, v_PID, v_Telephone, v_Address;

END LOOP;

-- 关闭游标
CLOSE customer_cursor;

END;

/
```

调用存储过程，如图3.3所示。

```
CALL usp_getWithoutTrade(
    '2023-01-01 00:00:00',
    '2024-08-08 23:59:59'
);
```

```
[NOTICE] 统计未发生交易的客户
[NOTICE] -----
[NOTICE] 客户人数: 6, 客户总余额: 1531.00
[NOTICE] 客户号: 7, 客户姓名: Robert, 身份证号: 110000000000000007, 电话: 13900000007, 地址: <NULL>
[NOTICE] 客户号: 11, 客户姓名: Emma, 身份证号: 110000000000000011, 电话: 13900000011, 地址: 辽宁省大连市
[NOTICE] 客户号: 8, 客户姓名: Jessica, 身份证号: 110000000000000008, 电话: 13900000008, 地址: <NULL>
[NOTICE] 客户号: 12, 客户姓名: Henry, 身份证号: 110000000000000012, 电话: 13900000012, 地址: <NULL>
[NOTICE] 客户号: 10, 客户姓名: Olivia, 身份证号: 110000000000000010, 电话: 13900000010, 地址: <NULL>
[NOTICE] 客户号: 9, 客户姓名: William, 身份证号: 110000000000000009, 电话: 13900000009, 地址: <NULL>
```

图3.3 调用存储过程3

7. 统计银行卡交易量和交易额

用存储过程统计银行卡交易量和交易额。

```
CREATE OR REPLACE PROCEDURE usp_getTradeInfo(
    IN date1 TIMESTAMP,
    IN date2 TIMESTAMP DEFAULT NULL,
    IN address VARCHAR(20) DEFAULT NULL
)
```

```
AS
DECLARE
    CNT1 INT;
    Total1 DECIMAL(18,2);
    CNT2 INT;
    Total2 DECIMAL(18,2);
BEGIN
    -- 设置默认的截止日期
    IF date2 IS NULL THEN
        date2 := CURRENT_TIMESTAMP;
    END IF;

    IF address IS NULL THEN
        -- 没有地址条件的查询
        SELECT COUNT(tradeMoney), SUM(tradeMoney)
        INTO CNT1, Total1
        FROM tradeInfo
        WHERE tradeDate BETWEEN date1 AND date2 AND tradeType='存入';

        SELECT COUNT(tradeMoney), SUM(tradeMoney)
        INTO CNT2, Total2
        FROM tradeInfo
        WHERE tradeDate BETWEEN date1 AND date2 AND tradeType='支取';
    ELSE
        -- 带地址条件的查询
        SELECT COUNT(tradeMoney), SUM(tradeMoney)
        INTO CNT1, Total1
        FROM tradeInfo JOIN cardInfo ON tradeInfo.cardID = cardInfo.cardID
        JOIN userInfo ON cardInfo.customerID = userInfo.customerID
        WHERE tradeDate BETWEEN date1 AND date2 AND tradeType='存入'
        AND address LIKE '%' || address || '%';

        SELECT COUNT(tradeMoney), SUM(tradeMoney)
        INTO CNT2, Total2
```

```
FROM tradeInfo JOIN cardInfo ON tradeInfo.cardID = cardInfo.cardID
      JOIN userInfo ON cardInfo.customerID = userInfo.customerID
WHERE tradeDate BETWEEN date1 AND date2 AND tradeType='支取'
      AND address LIKE '%' || address || '%';
END IF;

-- 输出统计结果
RAISE NOTICE '统计银行卡交易量和交易额';
RAISE NOTICE '-----';
RAISE NOTICE '起始日期: % 截止日期: % ', date1, date2;
RAISE NOTICE '-----';
RAISE NOTICE '存入笔数: % 存入金额: % ', CNT1, Total1;
RAISE NOTICE '支取笔数: % 支取金额: % ', CNT2, Total2;
RAISE NOTICE '-----';
RAISE NOTICE '发生笔数: % 结余金额: % ', CNT1 + CNT2, Total1 - Total2;
END;
/
```

调用存储过程，如图3.4所示。

```
CALL usp_gettradeinfo('2024-08-25', '2024-08-27', NULL);
```

```
[NOTICE] 统计银行卡交易量和交易额
[NOTICE] -----
[NOTICE] 起始日期: 2024-08-07 00:00:00 截止日期: 2024-08-09 00:00:00
[NOTICE] -----
[NOTICE] 存入笔数: 6 存入金额: 10300.00
[NOTICE] 支取笔数: 4 支取金额: 3500.00
[NOTICE] -----
[NOTICE] 发生笔数: 10 结余金额: 6800.00
```

图3.4 调用存储过程4

任务 4：使用事务进行转账

本实验的具体操作可参见视频。

用户John（卡号为1010357600000001）想要给用户Jane（卡号为

1010357600000002) 转账500元, 可以使用一个事务完成此项任务。

创建事务如下。

```
BEGIN ;

INSERT INTO tradeInfo (tradeType, cardID, tradeMoney, remark)
VALUES ('支取', '1010357600000001', 500, '转账扣款');

INSERT INTO tradeInfo (tradeType, cardID, tradeMoney, remark)
VALUES ('存入', '1010357600000002', 500, '转账收款来自John');

COMMIT;
```

查看交易信息表, 如图4.1所示。

12	2024-08-26 20:37:09	支取	1010357600000001	500.00	转账扣款
13	2024-08-26 20:37:13	存入	1010357600000002	500.00	转账收款来自John

图4.1 查看交易信息表结果

如果John转账5000元出现报错, 则需要ROLLBACK回滚。

```
ROLLBACK;
```

注意事项

- (1) 生成银行卡号中卡号唯一性判别。
- (2) 事务中Rollback的使用。

随堂测试

1. 有关存储过程的参数默认值，下面说法正确的是()。
 - A. 输入参数必须有默认值
 - B. 带默认值的输入参数，方便用户调用
 - C. 带默认值的输入参数，用户不能再传入参数，只能采用默认值
 - D. 输出参数也可以带默认值

2. 下面有关存储过程的说法，()是错误的。
 - A. 它可作为作为一个独立的数据库对象并作为一个单元供用户在应用程序中调用
 - B. 存储过程可以传入和返回(输出)参数值
 - C. 存储过程必须带参数，要么是输入参数，要么是输出参数
 - D. 存储过程提高了执行效率

3. 在openGauss触发器中，触发事件不包括()
 - A. INSERT
 - B. TRUNCATE

C. UPDATE

D. CHECK

4. 用于判断游标是否成功提取到数据的属性是()

A. %ROWCOUNT

B. ROWCOUNT

C. %FOUND

D. ISFOUND

5. 修改触发器的语句是()

A. CREATE TRIGGER

B. CREATE OR REPLACE TRIGGER

C. ALTER TRIGGER

D. MODIFY TRIGGER